

GALAXY GUARDIAN

Final Master Project Blueprint

Verified Troubleshooting + Evidence-Driven Device Diagnosis + Outcome-Aware Replanning

A website and REST API for Samsung Galaxy troubleshooting. The mandatory challenge-compatible plan engine works independently; an optional investigation layer makes supported plans more context-sensitive.

PROJECT LOCK	DESIGN RULE
Product	Website + backend REST service; not a mandatory Android app
Core promise	Understand → Investigate when useful → Retrieve → Plan → Compile → Verify → Act → Recheck
Safety boundary	Only supplied approved troubleshooting sources and exact catalog deeplinks authorize executable output
Reality boundary	A website cannot silently read battery, app, network or temperature telemetry

Document status: Consolidated implementation decision, not a claim that every proposed feature is in the organizer brief. Actual organizer schema.py, sample outputs and supplied asset content take precedence over illustrative examples.

Sources: supplied nine-page Theme 2 brief; previously supplied Galaxy Guardian blueprint; SmartGuided architecture and requirements described in this conversation.

1 | Product definition and non-negotiable scope

Galaxy Guardian turns a vague complaint into a supported, safely ordered troubleshooting plan with exact verified Settings catalog deeplinks. When multiple supported causes remain plausible, its investigator asks a small number of useful diagnostic questions, records evidence and chooses an approved next test. After a guided action it records the outcome and revises the next step if needed.

Mandatory challenge-compatible engine

- POST /v1/troubleshoot and GET /health; exact organizer request/response schema and fallback shapes.
- Normalize informal complaints and produce the required 8-10 meaning-preserving query variations.
- Retrieve/extract approved siiS troubleshooting material; output only reference-supported atomic actions.
- Map to exact supplied deeplink catalog entries; group steps by screen, split distinct screens.
- Order low-disruption steps first and critical/destructive actions last; honor exact category enums.
- Prevalidated semantic fast path; version-sensitive cache; benchmark step accuracy, latency and cost/query.
- Return strict official JSON from the official endpoint; no extra investigation UI fields in its response.

Differentiating, separately testable extension

- Evidence ledger, supported possible-cause graph and deterministic next-test selection.
- Battery-drain/overheating diagnostic walkthrough using only approved checks where the dataset supports them.
- Outcome-aware replanning and context-preserving escalation summary.
- Action compiler with explicit action polarity and target-control compatibility.
- Adversarial and contrastive evaluation, measured dashboard and an explanatory website.

Not claimed: automatic phone telemetry, autonomous settings changes, verified hardware root cause, unrestricted Samsung device access or real rollback. These require additional authorized integration and evidence.

2 | End-user journey and screens

SCREEN	WHAT THE USER DOES / SEES
1. Diagnose	Enter complaint; optionally choose Galaxy model, OS, recent update, charging state and other context.
2. Understanding	See detected symptoms and uncertainty; system determines whether evidence is sufficient for a direct plan.
3. Investigate (conditional)	Answer one high-value question or perform an approved diagnostic check; optionally enter an observed result.
4. Resolution plan	Ordered action cards with category, risk, exact catalog-based Open Setting where supported, Why this step? and manual guidance.
5. Recheck	Report improvement, unchanged condition or uncertainty after an appropriate observation window.
6. Continue / escalate	If unresolved, select a different eligible test or create a support summary preserving the evidence and attempted actions.
7. Engine metrics (demo)	Show real schema, grounding, deeplink, cache, latency and cost results; no fabricated percentages.

Illustrative battery journey

“Phone is hot and battery drains rapidly after update.” Detect overheating + drain + post-update context. Ask whether heat occurs while charging, while a particular app is active, when idle or during mobile data use—only if the answer changes an available approved next check. If the supplied corpus supports app battery usage, guide the user to that screen; if not, do not invent the step. Follow up after a meaningful interval and update the case.

3 | Architecture and control boundaries

```
React / TypeScript / Tailwind WEBSITE
| POST /v1/troubleshoot (official, stateless)
| /v1/investigations/* (optional, stateful)
v
FastAPI request validation + schema.py
|-> Stage 1: complaint normalization / intent / variations
|-> Version- and context-aware semantic cache -- HIT -> revalidate
|-> Hybrid retrieval: BM25 + vectors + fusion + reranking
|-> Approved evidence + source IDs
|-> Stage 2: source-constrained structured candidate planner
|-> Action compiler + exact catalog screen resolver
|-> Safety ordering + deterministic plan verifier
|-> PASS: official JSON + cache  FAIL: bounded repair/fallback
+--> Optional investigator: case -> ledger -> approved test
    -> observation -> shared plan engine -> outcome -> replan
```

Authority separation: LLMs interpret language and propose candidate structures. Approved references authorize steps. The action catalog owns deeplinks. Deterministic rules select approved diagnostics, enforce safety and authorize the final response. No model-generated URI or unsupported step reaches the user.

Official direct path versus investigation path

The direct plan endpoint never requires a case, chat history, a dashboard or telemetry. The investigation endpoints are a product extension and must not alter official request/response fields. Optional evidence can be provided through a supported request field or a controlled internal adapter, never by silently changing the official contract.

4 | Data foundation and knowledge model

- Ingest queries.json, siiS_responses.json, deeplinks.json, schema.py and supplied examples; keep raw immutable copies and version hashes.
- Parse source passages into source_id, domain, symptom/issue tags, goal, atomic step, action group, category and screen hints. Preserve exact source spans for provenance.
- Index lexical text with BM25 and semantic text with embeddings. Index catalog description, message and qa_description, not obfuscated URI tokens.
- Relational model: Source, Symptom, Issue, Evidence, Action, ActionGroup, SettingsControl, Deeplink, Constraint; optional Case, Observation, Hypothesis, DiagnosticTest, Outcome and CacheEntry.
- Audit actual coverage in Battery, Display, Camera and Performance before committing to a specific diagnostic branch. Build a source-to-action and test-to-source coverage matrix.

PostgreSQL + pgvector is the preferred shared persistence layer; a lightweight local index is acceptable for early tests. Redis is optional for prevalidated fast-path lookups. Do not add a graph database merely to call this a knowledge graph.

5 | Stage 1: complaint intelligence

Input: raw complaint plus permitted optional device context and supplied reference text. Output: normalized intent, symptom list, context, uncertainty/missing information and 8–10 meaning-preserving variations when the official contract requires them.

- Handle slang, typos, paraphrases, multi-symptom reports, timing (“after update”) and negative statements (“not charging”).
- Preserve polarity and uncertainty: “battery drains only while gaming” must not become “battery drains while idle.”
- Keep canonical intent separate from device-specific evidence and use both in cache compatibility checks.
- Use structured LLM output and Pydantic validation; fall back to deterministic normalization if model output is malformed.

6 | Retrieval and grounded planning

Run BM25 and embedding search in parallel on approved material; fuse ranked candidates and optionally rerank. Retrieve passages with source IDs and explicit coverage. Where a request supplies `siiS_response`, follow the organizer-defined precedence rather than replacing it with unrelated indexed text.

Stage 2 receives canonical intent, approved evidence, the exact schema and allowed action vocabulary. It extracts goals and atomic actions, but emits only a candidate plan. It must not fabricate references, URLs, screen IDs or deeplinks. If there is no supported material, return the exact official `no_match/no_siiS_context` fallback as appropriate.

Action compiler: the shared core differentiator

```
candidate action
-> supported source span?
-> catalog action/control exists?
-> correct verb and polarity? (open/enable/disable)
-> compatible screen and category?
-> copy EXACT masked catalog URI
-> split by distinct screen / merge same-screen steps
-> safe order -> strict verifier -> approved output
```

Example: “Open Touch Sensitivity” is not equivalent to “Enable Touch Sensitivity.” Semantic similarity alone cannot authorize a control operation. Manual physical interventions must not be given invented Settings deeplinks.

7 | Deterministic safety and hallucination firewall

- Exact `schema.py` validation: nesting, required fields, enums, lengths, goal/title/description wording, score, variations and fallback shape.
- Every emitted action has approved source support; every emitted deeplink is an exact catalog member and compatible with the action and screen.
- No external `http/https/www/Markdown` links or fabricated URIs in official plan output. Reserved generic fallback only under brief-defined conditions.
- Correct safe ordering, critical actions last, no duplicates or conflicting toggles, no unsafe action before required checks.
- One constrained candidate repair followed by full revalidation; otherwise official fallback or controlled error, never silent fabrication.

- Revalidate cached payloads against relevant current source, catalog and schema versions.

8 | Evidence-driven diagnostic investigator

The investigator is the major product extension. It should collect only evidence that can change the next supported action. It does not claim definitive root-cause diagnosis without appropriate observations and reference support.

Battery/overheating diagnostic dimensions

OBSERVATION	POSSIBLE USE	INPUT LIMITATION
App activity / usage	Check whether an approved app-usage test is relevant.	User-entered or authorized upstream data.
Idle vs active drain	Distinguish circumstances and choose a supported check.	A drain rate needs comparable timed observations.
Charging context	Separate charging-associated heat from other reports.	Ask user; no silent charging telemetry.
Signal / network	Evaluate an approved connectivity check if available.	User-reported or authorized signal data.
Software update timing	Retrieve relevant post-update procedures.	Reported update history; timing alone is not proof.
Temperature / heat pattern	Select an approved heat-related test and safety guidance.	Subjective reports are not calibrated sensor readings.

Hypothesis and test policy

Represent supported possible causes as hypotheses with supporting, contradicting and missing observations. Each approved diagnostic test has source_id, eligibility/preconditions, possible outcomes, distinguishing targets, effort, risk and expected observation. Reject unsupported, unsafe, redundant or unavailable tests. Select the test that distinguishes unresolved supported hypotheses, then prefer lower risk and effort with stable deterministic tie-breaking. Do not invent numerical confidence probabilities.

- complaint + provenance-tagged observations
- > supported hypotheses and missing evidence
- > eligible approved diagnostic tests
- > choose one informative, low-risk test
- > receive and validate observation
- > update evidence ledger
- > plan-ready? shared verified engine : next useful test
- > no eligible useful test? clarify / escalate

Outcome verification

Record action_id, baseline observation, follow-up observation, measurement conditions, observation window and result: improved, unchanged or inconclusive. Merely clicking Open Setting or marking an action completed never proves the device is fixed. If a fix fails, update the case and choose a different supported test or justified escalation.

9 | Semantic cache and determinism

Precompute embeddings for common intents and warm frequently validated plans. Cache only fully validated official responses. Compatibility key includes normalized intent, relevant device context/evidence signature, knowledge version, catalog version, schema version and planner version. A different observation must not incorrectly reuse a plan from a superficially similar complaint.

- Check exact/canonical match before approximate semantic match; apply a calibrated threshold and compatibility filters.
- Keep direct API latency independent of optional multi-turn investigation; measure cached P50/P95 and cold P50/P95 separately.
- Use pinned model/prompt versions, stable ranking and canonical serialization for reproducible outputs under the same source snapshot.

10 | REST API contracts

Official: POST /v1/troubleshoot; GET /health. Import organizer schema.py rather than reimplementing from this document. The following request is illustrative only:

```
POST /v1/troubleshoot
{
  "query": "phone overheating and battery drains after update",
  "siiS_response": "<optional supplied approved reference text>"
}
Response: EXACT organizer schema.py payload only.
```

Organizer-defined output fields referenced by the supplied blueprint include goal, title, score, actionName, description, steps, category, actionableDeeplink and query_variations. Confirm nesting, requiredness, exact enum values and fallback shape against the real schema.py. In particular, do not substitute internal labels such as auto for official category enums.

Optional extension endpoints (not organizer-mandated)

- POST /v1/investigations: create a case with complaint, optional device context and initial observations.
- GET /v1/investigations/{case_id}: current hypotheses, provenance-tagged evidence, next approved test and status.
- POST /v1/investigations/{case_id}/observations: submit approved test outcome or user observation.
- POST /v1/investigations/{case_id}/plans: request shared official plan engine when evidence is sufficient.
- POST /v1/investigations/{case_id}/outcomes: report after-action results and obtain next supported step.
- GET /v1/investigations/{case_id}/summary: downloadable/shareable support handoff without raw private logs.
- POST /v1/feedback: optional helpful/unhelpful and resolved/unresolved analytics.

Add case-level authorization, TTL/retention, input-size limits, server-side model credentials and request IDs. Keep personal identifiers out of the direct endpoint.

11 | Frontend implementation

React + TypeScript + Tailwind. Route flow: /diagnose → /investigate/:caseId (conditional) → /plan → /recheck → /summary; optional /metrics for judges. Use a typed API client generated from or checked against the backend contract. Official JSON is never reconstructed in the browser.

- Complaint entry: accessible multiline input, example chips, optional model/OS/update context; explicit consent for any additional observations.
- Processing: show real pipeline state only, not fake completed ticks; support timeout and fallback.
- Investigation: one high-value question at a time, Why this test?, source support, estimated effort and skip/not sure.
- Plan: ordered cards, category/risk badges, source-backed Why?, verified Settings action where supported, manual instructions where not.
- Recheck: after-action observation prompts, time-window reminders, improved/unchanged/inconclusive options.
- Escalation: concise problem, context, evidence provenance, tests attempted, actions completed and unresolved issue.
- Demo metrics: measured benchmark results and failure-blocking trace, never fabricated 98.x% numbers.

Deeplink reality: masked challenge deeplinks can be copied and validated against the catalog, but a normal website cannot guarantee opening native Settings on an actual Galaxy phone. Gate device launching on a tested supported client/integration; otherwise display the verified target and navigation guidance.

12 | Backend modules and repository

```
galaxy-guardian/  
  frontend/src/{pages,components,api,types,state}  
  backend/app/  
    api/          official and optional extension routes  
    schemas/      imported official contract + internal models  
    ingestion/    asset parsing and source versioning  
    enrichment/   Stage-1 complaint interpretation  
    retrieval/    BM25, embeddings, fusion, reranking  
    knowledge/    source and diagnostic graph relations  
    planner/      Stage-2 constrained extraction  
    actions/      atomic action compiler and polarity  
    deeplinks/    exact catalog resolver  
    safety/       deterministic sequencing  
    verifier/     schema, grounding, URI and conflict rules  
    investigator/ cases, ledger, test policy, outcomes  
    cache/        compatible prevalidated fast path  
    evaluation/   benchmarks, adversarial, contrastive  
    observability/ latency, cost, structured logs  
  main.py  
  data/raw/       original starter assets (unchanged)  
  data/processed/ versioned derived indexes  
  tests/{unit,integration,contract,adversarial,contrastive}  
  scripts/        ingest, audit, seed, benchmark  
  docker-compose.yml README.md AI_DISCLOSURE.md
```

13 | Recommended engineering stack

LAYER	CHOICE / REASON
Frontend	React + TypeScript + Tailwind; accessible, polished website.
API	FastAPI + Python; official Pydantic schema.py as source of truth.
Storage	PostgreSQL + pgvector; relational evidence and vector search.
Lexical retrieval	BM25 over approved references and catalog descriptive fields.
Models	Two logical LLM stages: normalization and constrained source extraction; provider configurable.
Cache	Pre-warmed in-process lookup, Redis where measured value warrants it.
Evaluation	pytest + fixture-driven contract, retrieval, safety and adversarial tests.
Deployment	Docker Compose, environment-based secrets, CI via GitHub Actions.

14 | Evaluation and acceptance

DIMENSION	TEST / METRIC
Contract	100% schema-conformant outputs on test corpus; exact fallbacks; zero external URL leakage.
Query	Canonical intent accuracy and 8–10 valid paraphrases without semantic drift.
Retrieval	Recall@K and MRR against labeled approved passages.
Plans	Step accuracy, supported-action precision/recall and source-span coverage.
Deeplinks	Exact catalog URI match; screen/control/polarity compatibility.
Safety	Critical-last rate; conflicting action rejection; unsupported action block rate.
Diagnosis	Correct approved next-test selection on labeled, evidence-differentiated cases.
Outcome	Correct improved/unchanged/inconclusive states; no false resolution on mere completion.
Speed	Cached P50/P95, cold P50/P95, cache hit rate, cost/query.
Robustness	Prompt injection, typos, multi-symptom, missing source, wrong URI, altered catalog.

Targets reported in the existing Galaxy Guardian blueprint for verification against organizer materials: 100% schema conformance, $\geq 80\%$ paraphrase cache hit rate, cached P95 ≤ 300 ms and cold P95 ≤ 8 s. Do not claim achieved results before measuring on a defined workload.

15 | Adversarial and contrastive test plan

- Slang, misspelling, ambiguous and multi-symptom complaints; contradictory user evidence.
- Model invents “Advanced Thermal Recovery Mode”: unsupported source/catalog action must be blocked.
- Correct screen but wrong polarity (open versus enable versus disable) must fail.
- Invalid/malicious URI, external URL and prompt injection inside retrieved text must fail.
- Duplicate actions, same-screen grouping errors, critical action at Step 1 and manual step with deeplink must fail.
- No source, no eligible diagnostic, inconclusive follow-up and LLM timeout must produce controlled outcomes.
- Contrastive: remove a catalog deeplink → affected action disappears or approved fallback; mark action unsupported → never emit; mark action critical → move last.
- Contrastive: same complaint, different supported observations → different approved next test; same complaint, same evidence snapshot → stable output.

16 | Build order, dependencies and scope gates

PHASE	DELIVERABLE / EXIT CRITERION
0. Asset audit	Read all supplied JSON/schema/examples; create coverage matrix; identify one supported battery investigation.
1. Contract + verifier	Exact official API model, strict validator and fallback fixtures pass.
2. Grounded core	Ingestion, BM25/vector retrieval, constrained planner, compiler and deeplink resolver pass end-to-end.
3. Fast path	Version-aware semantic cache; measured paraphrase and latency benchmarks.
4. Investigator MVP	One supported battery branch; ledger, 2+ eligible tests if references permit, outcome recording.
5. Website	Diagnose, conditional investigation, plan, recheck, escalation and metrics pages.
6. Hardening	Adversarial/contrastive tests, privacy, observability, Docker and CI.
7. Demo	Three reproducible scripted cases with real metrics and transparent simulated inputs.

Scope gate: If the supplied assets do not support a particular battery app/network branch, demonstrate it only as clearly labeled synthetic policy fixture—not as an organizer-approved repair. Preserve the official direct engine regardless.

17 | Three-part judge demo

- A. Supported normal path: messy complaint → normalized symptoms → retrieved source → exact verified catalog actions → schema-valid JSON.
- B. Actual diagnosis: same battery complaint in two labeled cases with different user/upstream observations → different eligible approved tests → one failed or inconclusive result → updated investigation.
- C. Reliability: paraphrase semantic cache hit with measured latency, then inject fabricated action or URI and show verifier rejection.

18 | Feature preservation checklist

- Official REST/schema, four domains, query variations, source-only extraction, atomic actions, exact masked deeplinks, safe ordering, semantic fast path.
- Hybrid retrieval, relational troubleshooting knowledge graph, two logical LLM stages, action compiler, polarity-aware matching, deterministic verifier and bounded repair.
- Battery overheating/drain investigation, app-activity/idle/charging/network/update context when supported, evidence provenance, approved test policy, outcomes and replanning.
- Website flow, Why this step?, explicit uncertainty, support escalation, feedback, analytics, cost and latency instrumentation.
- Adversarial suite, contrastive plan tests, stable output, dataset coverage audit, privacy controls, no fake telemetry and no fake metrics.

Final pitch: Galaxy Guardian investigates ambiguous Galaxy complaints using approved evidence, then compiles every recommended action against a trusted Settings catalog and checks whether the intervention helped—while preserving a fast, schema-exact REST troubleshooting service.

19 | Source and assumption register

Official baseline: supplied “Theme 2 – Troubleshooting / Smart Guided Troubleshooting Engine” nine-page PDF. Its schema.py and starter assets must be audited directly; exact field-level rules in this document are provisional where the complete schema is not supplied here.

Earlier design: “Galaxy Guardian Final Project Blueprint” (13 pages), especially official/optional separation, evidence ledger, deterministic test policy, provenance and outcome verification.

Consolidated design choice: SmartGuided action compiler, polarity checks, hybrid retrieval, explicit website and contrastive evaluation merged with Galaxy Guardian’s investigator. These are proposed team features, not automatically organizer requirements.